# Git & GitHub Version Control Interview Questions

A Focused Guide to Git & GitHub as a Version Control System

**Includes an "In Simple Terms" explanation under every answer**

**76 Questions** **4 Topic Areas** **Scenario-Based Included**

Compiled July 2026 · Covers core Git mechanics, branching/merging/rebasing, pull request & versioning workflows, and troubleshooting — no GitHub Actions or platform features

## Contents


1. Git Fundamentals & Core Concepts 25 questions
2. Branching, Merging & Rebasing 20 questions
3. Collaboration, Pull Requests & Versioning Practices 15 questions
4. Scenario-Based & Troubleshooting Questions 16 questions

# 1. Git Fundamentals & Core Concepts

**Q1. What is Git, and what problem does it solve?** Beginner

Git is a distributed version control system that tracks changes to files over time, letting multiple people work on the same project without overwriting each other's work. It solves the problem of losing history, coordinating parallel changes, and recovering earlier versions of a codebase.

**IN SIMPLE TERMS**

*Git is like a very detailed 'save history' for your project. Every change is recorded, so you can always go back, see who changed what, and combine work from several people without losing anything.*

**Q2. What is the difference between Git and GitHub?** Beginner

Git is the version control tool itself - it runs locally on your machine and manages history, branches, and commits. GitHub is a cloud-based hosting service built around Git that adds collaboration features like pull requests, issues, code review, and hosting for remote repositories.

**IN SIMPLE TERMS**

*Git is the engine that tracks changes. GitHub is a website that hosts that engine's output online and adds tools for teams to collaborate and review code.*

**Q3. What does it mean that Git is a 'distributed' version control system?** Beginner

Every clone of a Git repository contains the entire history of the project, not just a pointer to a central server. This means you can commit, branch, and view history entirely offline, and there's no single point of failure the way there is with older centralized systems like SVN.

**IN SIMPLE TERMS**

*Everyone's copy of the project has the complete history built in, not just a partial view. That means you can work and save your progress even without an internet connection.*

**Q4. What is a repository (repo) in Git?** Beginner

A repository is the full collection of files, folders, and their entire commit history for a project, tracked by Git. It can exist locally on a machine or remotely on a hosting service like GitHub.

**IN SIMPLE TERMS**

*A repo is just the project folder plus its entire history of changes, all bundled together and tracked by Git.*

**Q5. What is a commit?** Beginner

A commit is a saved snapshot of the repository's files at a specific point in time, along with a message describing the change and metadata like author and timestamp. Commits form the building blocks of a project's history.

**IN SIMPLE TERMS**

*A commit is like hitting 'save' with a note attached explaining what you changed - and Git keeps every single one of these saves forever.*

**Q6. What is the difference between git add and git commit?** Beginner

git add stages changes - moving them into a holding area to be included in the next commit. git commit then actually records those staged changes as a permanent snapshot in the project's history.

**IN SIMPLE TERMS**

*Adding is like putting items in a shopping basket; committing is like checking out and paying - only what's in the basket gets recorded.*

**Q7. What is the staging area (index)?** Beginner

The staging area is an intermediate space between your working directory and the repository's history, where you assemble exactly which changes will go into the next commit - even if you've modified more files than you want to commit at once.

**IN SIMPLE TERMS**

*It's a middle step where you choose exactly what goes into your next 'save,' even if you've changed a bunch of other things you're not ready to save yet.*

**Q8. What is the difference between the working directory, staging area, and repository?** Beginner

The working directory is the actual files on disk that you're editing. The staging area holds changes queued for the next commit. The repository (.git folder) stores the full committed history. Changes move through these three stages: edit, stage, commit.

**IN SIMPLE TERMS**

*Working directory = what you're currently editing. Staging area = what you've picked to save next. Repository = the permanent saved history.*

**Q9. What does git status show you?** Beginner

It shows the current state of the working directory and staging area: which files are modified, staged, untracked, or in conflict, relative to the last commit.

**IN SIMPLE TERMS**

*It's a quick summary telling you what's changed, what's ready to be saved, and what Git doesn't know about yet.*

**Q10. What is the difference between git diff and git diff --staged?** Beginner

git diff shows changes in the working directory that have not yet been staged. git diff --staged (or --cached) shows the changes that have been staged and will be included in the next commit.

**IN SIMPLE TERMS**

*Plain diff shows what you've changed but haven't queued up yet. Staged diff shows what you've already queued up to save.*

**Q11. What is a branch in Git?** Beginner

A branch is a lightweight, movable pointer to a specific commit, allowing you to diverge from the main line of development and work on something (a feature, a fix) without affecting other branches until you're ready to merge.

**IN SIMPLE TERMS**

*A branch is like a parallel timeline for your project - you can experiment or build a feature separately without messing up the main version until you're ready.*

**Q12. What is HEAD in Git?** Beginner

HEAD is a pointer that refers to the commit currently checked out in your working directory - typically the tip of whichever branch you're on. It moves automatically as you commit or switch branches.

**IN SIMPLE TERMS**

*HEAD marks 'you are here' in the project's history - it tells Git which commit or branch you're currently looking at and working from.*

**Q13. What is a detached HEAD state?** Beginner

It occurs when you check out a specific commit (rather than a branch), so HEAD points directly at that commit instead of at a branch tip. Any new commits made in this state aren't attached to a branch and can be lost once you switch away, unless you create a new branch from them first.

**IN SIMPLE TERMS**

*This happens when you 'time travel' to an old commit directly instead of a named branch. If you make changes there without creating a new branch, they can easily get lost.*

**Q14. What is the difference between git fetch and git pull?** Beginner

git fetch downloads new commits, branches, and tags from a remote repository into your local repo without touching your working directory or current branch. git pull does a fetch followed immediately by a merge (or rebase) into your current branch.

**IN SIMPLE TERMS**

*Fetch just checks for updates and downloads them quietly in the background. Pull downloads the updates and immediately merges them into what you're working on.*

**Q15. What is a remote in Git?** Beginner

A remote is a reference to a version of the repository hosted elsewhere (like on GitHub), typically named 'origin' by default, that you can fetch from or push to in order to sync work with others.

**IN SIMPLE TERMS**

*A remote is just the nickname Git uses for the copy of the project living somewhere else, like on GitHub, so you know where to send and receive updates.*

**Q16. What does git clone do?** Beginner

It creates a full local copy of a remote repository, including its entire history and all branches/tags, and automatically sets up that remote as 'origin' for future syncing.

**IN SIMPLE TERMS**

*It downloads a complete copy of a project - history and all - onto your machine, ready to work with.*

**Q17. What is a .gitignore file used for?** Beginner

It lists file and folder patterns that Git should never track or stage, such as build artifacts, dependency folders, or local environment files, keeping the repository clean of files that shouldn't be versioned.

**IN SIMPLE TERMS**

*It's a 'do not track these' list, so Git ignores things like temporary files, build outputs, or secrets that shouldn't be saved into the project's history.*

**Q18. What is the difference between git rm and simply deleting a file?** Beginner

Deleting a file from disk only removes it from the working directory; Git will still show it as tracked and missing. git rm deletes it and also stages that removal, so the deletion itself becomes part of the next commit.

**IN SIMPLE TERMS**

*Just deleting the file leaves Git confused about what happened. git rm tells Git 'yes, I meant to delete this, please record it.'*

**Q19. What is a Git hook?** Beginner

A hook is a script that Git automatically runs at specific points in the workflow - like before a commit, after a commit, or before a push - used to enforce checks such as linting, tests, or commit message formatting.

**IN SIMPLE TERMS**

*A hook is an automatic script that runs at key moments, like right before you commit, so you can catch mistakes (like bad formatting) before they get saved.*

**Q20. What is the difference between a lightweight tag and an annotated tag?** Beginner

A lightweight tag is just a named pointer to a specific commit, with no extra metadata. An annotated tag is a full Git object with its own message, author, and date, and can be GPG-signed - generally recommended for marking releases.

**IN SIMPLE TERMS**

*A lightweight tag is just a sticky note with a name on a commit. An annotated tag is a fuller record with a message and author info attached, better for marking official releases.*

**Q21. What is the SHA-1 (or SHA-256) hash used for in Git?** Beginner

Every commit, tree, and blob in Git is identified by a cryptographic hash of its contents, which uniquely fingerprints that object and lets Git detect any corruption or tampering, since even a tiny change produces a completely different hash.

**IN SIMPLE TERMS**

*Every single change gets its own unique fingerprint. If even one character changes, the fingerprint completely changes too, which is how Git can guarantee nothing was silently altered.*

**Q22. What is the difference between git log and git reflog?** Beginner

git log shows the commit history reachable from your current branch. git reflog shows a local record of everywhere HEAD has pointed recently - including commits from branches that were deleted or rewound - making it a safety net for recovering 'lost' work.

**IN SIMPLE TERMS**

*git log shows the official story of your project. git reflog is more like a personal browser history of everywhere you've been, even places that aren't part of the official story anymore - handy for undoing mistakes.*

**Q23. What are blobs, trees, and commits in Git's internal object model?** Beginner

A blob stores a file's raw content. A tree represents a directory, listing blobs and other trees with their names. A commit points to a tree (a full snapshot) plus metadata like the author and parent commit(s). Together they form Git's underlying data model.

**IN SIMPLE TERMS**

*Blob = a file's contents. Tree = a folder listing what's inside it. Commit = a labeled snapshot of an entire tree, with information about who made it and what came before.*

**Q24. What is a fork, and how is it different from a clone?** Beginner

A fork is a full copy of someone else's repository created on your own GitHub account, letting you make changes independently and later propose them back via a pull request. A clone is simply downloading a copy of any repository (yours or someone else's) to your local machine - it doesn't create a new copy on the server.

**IN SIMPLE TERMS**

*Forking makes your own separate copy of someone else's project on GitHub, so you can change it freely. Cloning just downloads any repo to your computer to work on locally.*

**Q25. What is the purpose of a README file?** Beginner

A README (usually README.md) is the first file most people see when visiting a repository. It typically explains what the project does, how to install and use it, and how to contribute - serving as the project's front door.

**IN SIMPLE TERMS**

*It's the welcome page for your project - the first thing people read to understand what it does and how to use it.*

## 2. Branching, Merging & Rebasing

**Q26. What is the difference between git merge and git rebase?** Intermediate

git merge combines two branches by creating a new merge commit that ties their histories together, preserving exactly what happened and when. git rebase replays your branch's commits on top of another branch's tip, creating new commits and producing a linear history, but rewriting commit hashes in the process.

**IN SIMPLE TERMS**

*Merge keeps both branches' full history and stitches them together with a 'joining' commit. Rebase rewrites your branch's commits as if they happened after the other branch's latest changes, making the timeline look like one straight line.*

**Q27. When would you choose merge over rebase, or vice versa?** Intermediate

Use merge to preserve the true history of how a shared branch evolved (especially for anything already pushed and shared with others). Use rebase to clean up your own local, not-yet-shared commits into a simpler linear history before merging - never rebase commits others have already pulled.

**IN SIMPLE TERMS**

*Merge is safer for shared or public history since it doesn't rewrite anything. Rebase is great for tidying up your own private work before sharing it, but risky once other people already have a copy of those commits.*

**Q28. What is a fast-forward merge?** Intermediate

When the branch being merged in is simply ahead of the current branch with no divergent commits, Git can just move the current branch's pointer forward to match - no new merge commit is created.

**IN SIMPLE TERMS**

*If nothing on your branch changed while the other branch moved ahead, merging is as simple as just catching your pointer up - no extra 'merge' commit needed.*

**Q29. What causes a merge conflict?** Intermediate

A conflict occurs when two branches have made different changes to the same lines of the same file (or one edited a file the other deleted), and Git can't automatically determine which version should win.

**IN SIMPLE TERMS**

*It happens when two people changed the exact same part of a file in different ways, and Git can't guess which change you actually want to keep.*

**Q30. How do you resolve a merge conflict?** Intermediate

Open the conflicted file and look for the conflict markers (<<<<<<<, =======, >>>>>>>); manually edit the file to keep the correct combination of changes, remove the markers, then stage the file with git add and complete the merge with git commit.

**IN SIMPLE TERMS**

*You open the file, see both versions marked out with special symbols, pick or blend the right content, delete those symbols, and tell Git you're done by staging and committing.*

**Q31. What does git rebase -i (interactive rebase) let you do?** Intermediate

It lets you rewrite recent local commit history: reordering commits, squashing several into one, editing commit messages, splitting commits, or dropping ones you don't want - all before that history is shared with others.

**IN SIMPLE TERMS**

*It's a tool to clean up your recent commits before anyone else sees them - combine messy ones together, reword messages, or remove ones you don't need.*

**Q32. What does 'squashing' commits mean?** Intermediate

Squashing combines multiple commits into a single commit, typically used to condense a messy series of work-in-progress commits into one clean, meaningful commit before merging into a shared branch.

**IN SIMPLE TERMS**

*It's taking a pile of small, messy 'save points' and merging them into one clean, well-described save point.*

**Q33. What is git cherry-pick used for?** Intermediate

It applies the changes from a specific commit on one branch onto another branch, without merging the entire branch - useful for grabbing a single bug fix or feature without bringing in unrelated history.

**IN SIMPLE TERMS**

*It lets you copy just one specific commit from another branch onto your current one, without pulling in everything else that branch has.*

**Q34. What is the difference between git reset and git revert?** Intermediate

git reset moves the current branch pointer backward (optionally altering the working directory/staging area too), effectively erasing commits from the branch's history - risky on shared branches. git revert creates a brand-new commit that undoes the changes of a previous commit, preserving history - safe for shared branches.

**IN SIMPLE TERMS**

*Reset erases history as if it never happened - risky if others already have that history. Revert adds a new commit that cancels out an old one, keeping the record of what happened intact - much safer to share.*

**Q35. What are the three modes of git reset (--soft, --mixed, --hard)?** Intermediate

--soft moves the branch pointer but leaves staged changes and the working directory untouched. --mixed (the default) moves the pointer and unstages changes but keeps them in the working directory. --hard moves the pointer and discards all staged and working directory changes entirely.

**IN SIMPLE TERMS**

*Soft: undo the 'save,' but keep everything ready to re-save. Mixed: undo the save and un-stage things, but keep your edits. Hard: undo the save and throw away the changes completely - the most destructive option.*

**Q36. What is git stash used for?** Intermediate

It temporarily shelves uncommitted changes (both staged and unstaged) so you get a clean working directory - useful when you need to quickly switch branches or pull updates without committing half-finished work.

**IN SIMPLE TERMS**

*It's a temporary shelf for your unfinished changes, so you can quickly switch to something else and come back to exactly where you left off later.*

**Q37. What is the difference between git stash pop and git stash apply?** Intermediate

git stash apply reapplies the most recent stashed changes but keeps them in the stash list. git stash pop reapplies them and then removes that entry from the stash list.

**IN SIMPLE TERMS**

*Apply brings your shelved changes back but keeps a backup copy on the shelf. Pop brings them back and clears them off the shelf.*

**Q38. What is a tracking branch?** Intermediate

A tracking branch is a local branch configured to be linked with a specific remote branch, so commands like git push and git pull know which remote branch to sync with by default without you specifying it every time.

**IN SIMPLE TERMS**

*It's a local branch that 'remembers' which remote branch it's connected to, so you don't have to type out the connection every single time you push or pull.*

**Q39. What does git branch -d do versus git branch -D?** Intermediate

git branch -d deletes a branch only if it has already been fully merged elsewhere, protecting you from losing unmerged work. git branch -D force-deletes the branch regardless of merge status, which can permanently lose commits if you're not careful.

**IN SIMPLE TERMS**

*Lowercase -d is the safe delete - Git refuses if you'd lose unmerged work. Uppercase -D is the forceful delete - it'll happily throw away unmerged work if you tell it to.*

**Q40. What is git bisect, and when would you use it?** Intermediate

git bisect performs an automated binary search through commit history to find the exact commit that introduced a bug. You mark a known-good and known-bad commit, and Git checks out commits in between for you to test, narrowing down the culprit in log2(n) steps.

**IN SIMPLE TERMS**

*It's a smart way to hunt down which commit broke something - you tell Git 'this one worked, this one didn't,' and it keeps testing points in between until it pinpoints the exact commit that caused the problem.*

**Q41. What is the 'golden rule' of rebasing, and why does it matter?** Advanced

Never rebase commits that have already been pushed and that other people may have based work on. Rebasing rewrites commit hashes, so anyone who already pulled the old commits will end up with a confusing, duplicated, or conflicting history when they try to sync afterward.

**IN SIMPLE TERMS**

*Don't rewrite history that other people have already downloaded - it creates a confusing mess for them when they try to sync up with your changed version.*

**Q42. What is a merge strategy, and what are some common ones?** Advanced

A merge strategy controls how Git combines diverging histories. Common ones include the default 'recursive'/'ort' strategy for typical two-branch merges, 'octopus' for merging more than two branches at once, and 'ours'/'theirs' to resolve conflicts by favoring one side entirely.

**IN SIMPLE TERMS**

*It's the method Git uses to figure out how to combine two branches. Most of the time it's automatic, but you can tell it special rules, like 'always favor my version' when conflicts happen.*

**Q43. What is a rerere (reuse recorded resolution), and when is it useful?** Advanced

git rerere records how you resolved a merge conflict so that if the exact same conflict appears again (common during repeated rebases or long-lived branches), Git can automatically reapply the same resolution instead of asking you to redo it.

**IN SIMPLE TERMS**

*It's Git remembering how you fixed a conflict before, so if the exact same conflict comes up again later, it can just reuse your old fix automatically instead of bothering you again.*

**Q44. What is the difference between a merge commit and a fast-forward commit in terms of history shape?** Advanced

A merge commit has two (or more) parent commits and visibly shows where two lines of development joined, creating a branching graph shape in history. A fast-forward simply advances a pointer with no new commit, keeping history perfectly linear as if the branch never existed.

**IN SIMPLE TERMS**

*A merge commit leaves a visible 'joining point' in the history graph. A fast-forward leaves no trace at all - the history just looks like one straight line.*

**Q45. How would you split a large commit into several smaller, logical commits?** Advanced

Use an interactive rebase to mark the commit for 'edit,' then use git reset HEAD~1 (or git reset --soft) to unstage its changes while keeping them in the working directory, and re-stage and commit them in smaller, logical chunks using git add -p for partial staging.

**IN SIMPLE TERMS**

*You reopen the big commit, un-save it back into your working files, then re-save it piece by piece in smaller, more meaningful chunks instead of one giant blob.*

## 3. Collaboration, Pull Requests & Versioning Practices

**Q46. What is a pull request (PR)?** Beginner

A pull request is a GitHub feature that proposes merging changes from one branch (often on a fork) into another, providing a dedicated space to review the diff, discuss changes, and formally approve before merging.

**IN SIMPLE TERMS**

*A pull request is like saying 'here's the change I made, please review it before we add it to the main project.' It's the review-and-discuss step before merging.*

**Q47. What is the difference between a pull request and a merge?** Beginner

A pull request is the proposal and review process - a conversation around a proposed change. A merge is the actual git operation of combining one branch's commits into another, which typically happens once a pull request is approved.

**IN SIMPLE TERMS**

*The pull request is the conversation and approval step. The merge is the actual technical action of combining the code, which happens once everyone's happy.*

**Q48. What is the typical fork-and-pull-request workflow for contributing to open source?** Intermediate

You fork the repository to your own account, clone your fork locally, create a branch for your change, commit and push it to your fork, then open a pull request from your fork's branch back to the original repository for the maintainers to review.

**IN SIMPLE TERMS**

*Copy the project to your account, make your changes there, then ask the original project's owners to pull your changes in via a pull request - all without needing direct write access to their repo.*

**Q49. What is a draft pull request?** Intermediate

A draft PR signals that a change is still a work in progress and isn't ready for formal review or merging yet, while still letting collaborators see progress and leave early feedback.

**IN SIMPLE TERMS**

*It's a way of saying 'here's my work in progress, feel free to look, but it's not ready to merge yet.'*

**Q50. What is a branch protection rule?** Intermediate

Branch protection rules let repository admins enforce requirements on important branches - such as requiring passing checks, a minimum number of approving reviews, or disallowing force-pushes and deletions - before changes can be merged.

**IN SIMPLE TERMS**

*It's a set of guardrails on important branches (like main) - for example, requiring someone to approve and preventing risky commands like force-push before any change can get in.*

**Q51. What is the difference between 'Squash and merge,' 'Rebase and merge,' and 'Merge commit' options on GitHub?** Intermediate

'Merge commit' preserves every individual commit and adds a merge commit tying them together. 'Squash and merge' combines all the PR's commits into a single commit on the target branch. 'Rebase and merge' replays the PR's individual commits directly onto the target branch with no merge commit, keeping history linear.

**IN SIMPLE TERMS**

*Merge commit keeps every little commit plus a 'joining' commit. Squash and merge boils the whole PR down into just one clean commit. Rebase and merge keeps the individual commits but lines them up neatly with no extra joining commit.*

**Q52. What is a merge conflict during a pull request, and how would you resolve it on GitHub?** Intermediate

It happens when the PR branch and the target branch have diverging changes to the same lines. You resolve it either using GitHub's web conflict editor for simple cases, or by pulling the target branch into your local branch, fixing conflicts locally, and pushing the resolution back up.

**IN SIMPLE TERMS**

*Same idea as a normal merge conflict - GitHub just shows it to you inside the PR page, and you either fix it right there in the browser or on your own machine and push the fix back up.*

**Q53. What is a monorepo, and what are its trade-offs compared to multiple repos?** Advanced

A monorepo stores multiple projects or services in a single repository, simplifying cross-project changes and dependency management, at the cost of a larger repo to clone and potentially broader blast radius for changes.

**IN SIMPLE TERMS**

*A monorepo puts everything in one giant project folder, which makes sharing code between parts easier, but it can get slower and require more careful handling as it grows.*

**Q54. What is 'trunk-based development,' and how does it differ from long-lived feature branches (e.g. GitFlow)?** Advanced

Trunk-based development has everyone merging small, frequent changes directly into a single shared branch (often behind feature flags), minimizing long-lived branches and merge conflicts. GitFlow-style workflows use longer-lived feature, develop, and release branches with more structured, less frequent merges.

**IN SIMPLE TERMS**

*Trunk-based means everyone commits small changes straight into the main line frequently, hiding unfinished features behind toggles. GitFlow-style means people work on separate long branches for longer before merging, with more formal steps in between.*

**Q55. What are conventional commits, and why do some teams use them?** Intermediate

Conventional Commits is a lightweight commit message convention (e.g., feat:, fix:, chore:) that makes history more readable and enables tooling to automatically generate changelogs and determine semantic version bumps based on commit type.

**IN SIMPLE TERMS**

*It's a standard format for writing commit messages (like starting with 'fix:' or 'feat:') so tools can automatically build changelogs and figure out version numbers based on what kind of change was made.*

**Q56. What is semantic versioning (SemVer), and how does it relate to Git tags?** Intermediate

SemVer uses a MAJOR.MINOR.PATCH version format where major bumps signal breaking changes, minor signals backward-compatible new features, and patch signals bug fixes. Git tags (often annotated) are commonly used to mark exactly which commit corresponds to each released version.

**IN SIMPLE TERMS**

*It's a version-numbering convention (like 2.4.1) where each part tells you something specific: the first number means big breaking changes, the second means new features, and the third means small fixes. Git tags mark exactly which commit was that version.*

**Q57. What best practices would you recommend for writing good commit messages?** Advanced

Keep the first line short (under ~50 characters) and written in the imperative mood ('Add feature' not 'Added feature'), leave a blank line, then add more detail in the body if needed explaining why the change was made, not just what changed.

**IN SIMPLE TERMS**

*Keep the headline short and to the point, written like a command ('Fix login bug' rather than 'Fixed the login bug'), and use the body underneath to explain why you made the change if it's not obvious.*

**Q58. What is Git LFS (Large File Storage), and when would you use it?** Advanced

Git LFS replaces large binary files (images, videos, datasets) in a repository with lightweight text pointers, while storing the actual file content separately, keeping the main repository small and fast to clone even when the project includes large assets.

**IN SIMPLE TERMS**

*Regular Git struggles with huge files like videos or datasets. LFS stores those big files separately and just keeps a small 'pointer' in your normal repo, so cloning stays fast.*

**Q59. How would you structure a Git branching strategy for a team shipping to production multiple times a day?** Advanced

A trunk-based approach works well here: short-lived feature branches merged frequently into a single main branch, gated by strong automated testing and feature flags to hide incomplete work, rather than long-lived release branches that accumulate risk and merge conflicts.

**IN SIMPLE TERMS**

*For teams shipping constantly, it's usually better to merge small changes into one main branch very often (hiding unfinished features behind toggles) rather than maintaining long-lived separate branches that get harder to merge the longer they exist.*

**Q60. What is 'Git flow,' and why has its popularity declined for many teams?** Advanced

Git flow is a branching model with dedicated long-lived develop, feature, release, and hotfix branches, originally designed for projects with scheduled releases. Many teams have moved away from it toward trunk-based development because its overhead doesn't fit well with fast, continuous deployment practices.

**IN SIMPLE TERMS**

*Git flow is an older, more structured branching system with lots of separate branch types. It made sense for slower, scheduled releases, but many teams now ship so frequently that its extra structure just gets in the way.*

## 4. Scenario-Based & Troubleshooting Questions

**Q61. You accidentally committed a secret (API key) and already pushed it to GitHub. What do you do?** Scenario

Treat the secret as compromised immediately and rotate/revoke it at the source (the service that issued it) - simply removing it from the repo isn't enough since it already exists in history. Then use a tool like git filter-repo or BFG Repo-Cleaner to strip it from the entire history, force-push the cleaned history, and notify collaborators to re-clone or carefully re-sync.

**IN SIMPLE TERMS**

*First, kill the leaked key at the source so it's useless even if someone has it - that matters more than cleaning the repo. Then use a special tool to scrub it out of history everywhere and let your team know to refresh their copies.*

**Q62. You committed to the wrong branch by mistake. How do you move that commit to the correct branch?** Scenario

Create (or switch to) the correct branch at the same point, then use git cherry-pick <commit-hash> to apply that commit there. Afterward, switch back to the original branch and remove the mistaken commit with git reset --hard HEAD~1 (if not yet pushed) or git revert if it's already shared.

**IN SIMPLE TERMS**

*Copy the commit over to the right branch using cherry-pick, then go back and remove it from the wrong branch - carefully, using revert instead of reset if you've already pushed it anywhere.*

**Q63. You're in a merge conflict and want to abort and start over. How?** Scenario

Run git merge --abort (or git rebase --abort if it happened during a rebase), which restores your branch to the state it was in right before the merge/rebase attempt began, discarding any partial conflict resolution.

**IN SIMPLE TERMS**

*There's a built-in 'never mind' command that puts everything back exactly how it was before you started the merge or rebase, so you can try again fresh.*

**Q64. Your local branch has diverged significantly from the remote main branch, and pulling causes a huge tangle of conflicts. What's a cleaner way to update?** Scenario

Rebase your branch onto the latest main (git fetch origin && git rebase origin/main) so your commits replay cleanly on top of the newest history, resolving conflicts commit-by-commit rather than all at once in one giant merge conflict.

**IN SIMPLE TERMS**

*Instead of merging everything at once (which dumps all conflicts on you together), rebase your work on top of the latest changes so you resolve any conflicts one small commit at a time, which is usually much easier to manage.*

**Q65. You need to undo the last commit, but you've already pushed it to a shared branch. What's the safe way to undo it?** Scenario

Use git revert <commit-hash>, which creates a new commit that cancels out the changes rather than rewriting history. This is safe for shared branches since nobody else's history gets invalidated, unlike git reset which would require a force-push.

**IN SIMPLE TERMS**

*Add a new commit that undoes the old one, rather than deleting or rewriting the old commit - that way nobody else's copy of the branch gets broken.*

**Q66. A teammate force-pushed over your commits on a shared branch and you lost work locally. How might you recover it?** Scenario

Check your local git reflog, which tracks every place HEAD has pointed on your machine, including the commits that got orphaned by the force-push; you can then create a new branch from the relevant reflog entry to recover the lost commits.

**IN SIMPLE TERMS**

*Your machine keeps a private history of everywhere you've been, even commits that got wiped out remotely. You can dig through that private history and recover your lost work from there.*

**Q67. You need to find out which commit introduced a specific bug across hundreds of commits. What's an efficient approach?** Scenario

Use git bisect: mark a known-good commit and the current known-bad commit, and Git will check out commits in between for you to test one at a time, narrowing down the exact commit that introduced the bug in a small number of steps.

**IN SIMPLE TERMS**

*Rather than checking every single commit, use the built-in 'binary search through history' tool - tell Git one point that worked and one that didn't, and it narrows down the exact broken commit quickly.*

**Q68. You want to combine the last five messy work-in-progress commits into one clean commit before opening a pull request. How?** Scenario

Run git rebase -i HEAD~5, then mark the last four commits as 'squash' (or 'fixup') in the editor that opens, and write a single clean commit message for the combined result.

**IN SIMPLE TERMS**

*Use interactive rebase to select the last several commits and tell Git to squish them together into one, then give that combined commit a clear, clean message.*

**Q69. A large binary file was accidentally committed months ago and is bloating the repository's size. How do you fix it?** Scenario

Removing the file in a new commit doesn't shrink the repo, since old versions remain in history. You need to rewrite history with git filter-repo (or BFG Repo-Cleaner) to strip the file from every commit, then force-push and have collaborators re-clone or carefully resync.

**IN SIMPLE TERMS**

*Just deleting the file now doesn't help - it's still sitting in old history taking up space. You need a special history-rewriting tool to remove it everywhere, then have everyone refresh their copy of the repo.*

**Q70. Two pull requests both modify the same file and the first one just got merged. What happens to the second PR, and what should the author do?** Scenario

GitHub will typically show the second PR as having a merge conflict or, in some cases, still show it as mergeable if the changes don't literally overlap on the same lines. The author should pull the latest target branch into their branch (merge or rebase), resolve any conflicts locally, and push the update.

**IN SIMPLE TERMS**

*The second PR usually needs to catch up with the changes that just got merged. The author brings in the latest version of the target branch, fixes any overlapping changes, and pushes the fix.*

**Q71. A pull request shows as 'This branch has conflicts that must be resolved' but you're not sure why, since you didn't touch those files. What's likely going on?** Scenario

Someone else likely merged changes into the target branch that touch the same lines your branch modified (or renamed/moved a file your branch also changed), even if you didn't edit that exact file in your latest commits - your branch is just out of date relative to main.

**IN SIMPLE TERMS**

*Most likely, the target branch (like main) moved forward with changes that overlap yours, even in a file you didn't recently touch. Your branch just needs to catch up with what's changed.*

**Q72. You need to permanently delete an old, sensitive file from a public repository's entire history, including all forks. What are the real limitations here?** Scenario

You can rewrite your own repository's history with git filter-repo and force-push, but you cannot force changes onto other people's forks or any caches/clones already made - if the file was ever public, you must assume copies may exist elsewhere and the underlying secret should be rotated regardless.

**IN SIMPLE TERMS**

*You can clean your own repo's history, but you can't reach into everyone else's existing forks or downloaded copies. If something sensitive was ever public, treat it as compromised and change the actual secret, don't just rely on deleting it from history.*

**Q73. How would you recover a deleted branch that was never merged, a week after it was deleted?** Scenario

Check git reflog locally if anyone still has a local copy that pointed at it recently, or check GitHub's own event history/API for the deleted branch's last commit SHA (GitHub often retains unreachable commits for a period), then recreate the branch pointing at that commit if it's found.

**IN SIMPLE TERMS**

*First check if anyone's local machine still remembers it (via reflog). If not, GitHub itself often keeps deleted commits around for a while behind the scenes, so there may still be a way to recover the exact commit and rebuild the branch from it.*

**Q74. A contributor's pull request includes commits with an incorrect email address that doesn't match their GitHub account, so it's not credited to their profile. How would they fix this for future commits?** Scenario

They should update their global Git config (git config --global user.email) to match an email address verified on their GitHub account, so future commits are properly attributed; already-pushed commits would need history rewriting (git rebase and amending) to fix retroactively, which is generally only worth doing before merging.

**IN SIMPLE TERMS**

*They just need to set the correct email in their Git settings going forward. Fixing already-made commits requires rewriting history, which is really only worth doing before the PR is merged.*

**Q75. Your organization wants to make sure no one can force-push or delete the main branch, even accidentally. How do you enforce that?** Scenario

Set up a branch protection rule on main that disables force-pushes and branch deletion, and typically also require pull request reviews and passing status checks before merging - enforced at the platform level so it applies regardless of individual permissions.

**IN SIMPLE TERMS**

*You turn on a protection setting for the main branch that simply blocks force-pushes and deletion outright, no matter who's trying to do it - a safety net built into GitHub itself rather than relying on people remembering not to.*

**Q76. You need to test whether a hotfix works correctly before merging it, without disturbing your team's ongoing feature work. What's a safe way to do this?** Scenario

Create a short-lived branch off of the affected release/production branch specifically for the hotfix, test and validate it in isolation, and merge it directly into that branch (and back into main) once confirmed - without touching any in-progress feature branches.

**IN SIMPLE TERMS**

*Make a small, separate branch just for the emergency fix, test it on its own, and merge it in once it's confirmed working - without touching or disrupting anyone else's in-progress feature branches.*